

实验二: 上下文无关文法与下推自动机

- 1. 设计上下文无关文法的变换算法
- 2. 构造与下推自动机等价的上下文无关文法

一、小组成员信息

姓名	班级	学号	分工
刘家辰	2024211302	2024210949	负责 CFG 输入解析模块设计，实现产生式读取、空串识别、文法存储等功能并完成运行测试。
张宇廷	2024211302	2024210948	负责 CFG 输入解析模块设计，实现产生式读取、空串识别等相关功能以及程序演示视频录制。
赵可	2024211302	2024210942	负责 PDA 转换 CFG 模块实现，包括状态变量构造、转移函数转换以及等价 CFG 生成等功能。
陈佳茵	2024211302	2024210967	负责 PDA 转换 CFG 模块实现以及程序整体框架整合，并完成实验报告整理与测试结果分析。

二、实验环境

本实验采用 C 语言进行程序设计与实现，程序主要使用了 `stdio.h`、`string.h`、`ctype.h` 和 `locale.h` 等标准库。

操作系统: Windows 11

开发工具: Visual Studio Code

编译环境: GCC / MinGW

编程语言: C 语言

程序文件: `transform.c`

三、实验目的

(1) 上下文无关文法的化简。

上下文无关文法的产生式右部形式比较自由，虽然表达能力较强，但在语法分析和自动机转换过程中可能会带来不便。因此，需要通过文法变换，在不改变文法语言生成能力的前提下，消除文法中的 ϵ 产生式、单产生式和无用符号。实验要求编程实现这些算法，并输出与原文法等价的化简后文法。

(2) 下推自动机到上下文无关文法的转换。

下推自动机和上下文无关文法都是描述上下文无关语言的重要工具。对于给定的

下推自动机，可以构造一个与其接受语言等价的上下文无关文法。本实验要求实现 PDA 到 CFG 的转换算法，并将转换得到的文法继续作为输入，调用前面实现的文法化简算法，得到消除 ϵ 产生式、单产生式和无用符号后的文法。

四、实验内容与程序功能

根据实验要求，本程序实现了上下文无关文法化简和下推自动机转换上下文无关文法两个核心功能。同时，为了增强程序的可测试性和通用性，代码中设计了四个菜单选项。程序运行后首先输出功能菜单，用户可以根据需要选择不同实验功能。主菜单包括输入 CFG 并化简、使用给定 CFG 样例、给定 PDA 转换 CFG 并化简、输入 PDA 转换 CFG 并化简四个选项。

- **选项 1 是手动输入 CFG 并化简：**用户先输入产生式行数，然后逐行输入上下文无关文法产生式，程序会解析输入内容，并依次完成消除 ϵ 产生式、消除单产生式和消除无用符号的操作。
- **选项 2 是使用实验要求中的给定 CFG 样例：**程序内部已经写入实验文档中要求验证的 CFG 样例，用户选择该选项后，无需手动输入文法，程序会直接对该样例进行化简，用于验证 CFG 化简算法的正确性。
- **选项 3 是将实验要求中的给定 PDA 转换为 CFG 并化简：**程序内部写入了实验文档给出的下推自动机转移规则，程序会先根据 PDA 构造等价的上下文无关文法，然后继续调用 CFG 化简算法，输出各阶段结果。
- **选项 4 是手动输入 PDA 并转换为 CFG：**该功能是在实验基本要求上的扩展，用户可以输入状态数、栈符号数、栈符号、初始状态、初始栈符号以及转移函数，程序根据输入内容构造等价 CFG，并继续进行文法化简。

因此，本程序不仅完成了实验要求中的固定样例验证，还支持自定义 CFG 和自定义 PDA 输入，具有一定的扩展性。

五、程序总体设计

本程序整体上可以分为五个模块：文法数据结构模块、CFG 输入解析模块、CFG 化简模块、PDA 转 CFG 模块和主函数交互模块。

1. **文法数据结构模块**用于统一保存文法信息。程序中定义了 Production 结构体和 Grammar 结构体。Production 结构体用于保存一条产生式，其中 lhs 表示产生式左部，rhs 表示产生式右部。Grammar 结构体用于保存整个文法，包括开始符号、非终结符集合、产生式数组和产生式数量。
2. **CFG 输入解析模块**负责读取和处理用户输入的产生式。程序可以识别 $\rightarrow$ 和 $\Rightarrow$ 两种箭头形式，也可以识别 | 分隔的多个候选式。同时，程序支持使用 #、 ϵ 、eps 等形式表示空串，提高了输入的灵活性。
3. **CFG 化简模块**是本程序的核心模块之一。该模块依次调用消除 ϵ 产生式、消除单产生式和消除无用符号三个算法。程序中的 simplify() 函数体现了这一执行顺序，即先输出消除 ϵ 产生式后的文法，再输出消除单产生式后的文法，最后输出消除无用符号后的文法。
4. **PDA 转 CFG 模块**用于根据下推自动机的状态、栈符号和转移函数构造等价上下文无关文法。程序中既提供了实验要求给定 PDA 样例的转换函数，也提供了自定义 PDA 输入和转换函数。对于 PDA 中的变量，程序采用类似 [qi, 栈符

- 号, qj]的思想表示推导过程,并在运行时输出变量含义,方便理解转换结果。
5. **主函数交互模块**负责显示菜单、读取用户选择,并根据不同选项调用对应功能。通过该设计,程序结构清晰,实验验证过程也更加直观。

六、核心算法设计

6.1 CFG 输入与解析算法

程序先对输入的产生式进行预处理,去掉空格,再识别产生式左部、箭头和右部。对于右部存在多个候选式的情况,程序使用|进行拆分,并分别保存为多条产生式。

核心代码如下:

```
alt = strtok(rhs, "|");
while (alt != NULL) {
    add_prod(g, lhs, is_epsilon(alt) ? "" : alt);
    alt = strtok(NULL, "|");
}
```

其中, is_epsilon()用于判断右部是否为空串。程序支持#、 ϵ 、eps 等多种空串输入形式,因此输入更加灵活。

6.2 消除 ϵ 产生式算法

消除 ϵ 产生式时,程序首先计算所有可空非终结符。如果某个非终结符可以直接推出 ϵ , 或者它的右部所有符号都可以推出 ϵ , 则该非终结符被标记为可空。

核心判断如下:

```
if (id >= 0 && !nullable[id] &&
    rhs_all_nullable(g, g->p[id].rhs, nullable)) {
    nullable[id] = 1;
}
```

得到可空符号后,程序对产生式右部进行递归处理。对于可空符号,既可以保留,也可以删除,从而生成新的等价产生式。这样可以在删除 ϵ 产生式的同时,尽量保持原文法的语言生成能力。

6.3 消除单产生式算法

单产生式是指形如 $A \rightarrow B$ 的产生式。程序先判断某条产生式是否为单产生式:

```
return strlen(p->rhs) == 1 && is_nt(g, p->rhs[0]);
```

然后程序建立非终结符之间的可达关系。如果 A 可以通过若干条单产生式到达 B, 就把 B 的非单产生式补充给 A, 从而删除中间的单产生式。

核心处理如下:

```
if (unit[i][j] && g->p[k].lhs == g->nt[j] &&
    !is_unit_prod(g, &g->p[k])) {
    add_prod(&out, g->nt[i], g->p[k].rhs);
}
```

这样处理后,文法中不再需要通过 $A \rightarrow B$ 这类单产生式进行间接推导。

6.4 消除无用符号算法

无用符号主要包括两类：不能推出终结字符串的符号，以及从开始符号出发无法到达的符号。

程序先判断符号是否具有生成能力：

```
if (!gen[lhs_id] && rhs_generating(g, g->p[i].rhs, gen)) {  
    gen[lhs_id] = 1;  
}
```

然后从开始符号出发，继续判断哪些非终结符是可达的：

```
if (id >= 0 && !reach[id]) {  
    reach[id] = 1;  
}
```

最后，程序只保留既能生成终结字符串、又能从开始符号到达的产生式。这样可以删除对文法语言没有实际贡献的部分。

6.5 PDA 转换 CFG 算法

PDA 转换 CFG 时，程序用一个变量表示“从某个状态出发，处理某个栈顶符号，并最终到达另一个状态”的过程。代码中通过 `var_name()` 函数生成变量名：

```
return (char)('A' + p * 4 + stack * 2 + q);
```

对于实验给定的 PDA，程序直接写入转移函数。例如：

```
add_push2(g, 0, 'b', 1, 0, 0, 1);
```

```
add_pop(g, 0, "a", 0, 1);
```

其中，`add_push2()` 用于处理压入两个栈符号的情况，`add_pop()` 用于处理弹栈情况。生成 CFG 后，程序继续调用前面的化简算法，对转换得到的文法进行消除 ϵ 产生式、消除单产生式和消除无用符号处理。

6.6 四个选项的整体调用逻辑

程序主函数根据用户输入的 `choice` 选择不同功能：

```
if (choice == 1) read_cfg(&g);  
else if (choice == 2) sample_cfg(&g);  
else if (choice == 3) pda_cfg(&g);  
else if (choice == 4) read_custom_pda_cfg(&g);
```

四个选项虽然输入来源不同，但最终都会得到一个 CFG。之后程序统一执行：

```
print_grammar(&g, "Original grammar:");  
simplify(&g);
```

因此，程序整体流程可以概括为：先根据用户选择获得文法，再统一进行文法化简。

七、程序输入输出格式

7.1 CFG 输入格式

当用户选择选项 1 时，程序首先提示输入产生式行数，然后逐行输入产生式。产生式左部一般为单个大写字母，右部可以是终结符、非终结符或空串。多个候选式之间使用 `|` 分隔，空串可以使用 `#` 表示。

示例输入格式如下：

Input number of production lines: 3

$S \rightarrow a|bA|\#$

$A \rightarrow abB|\#$

$B \rightarrow aA$

该输入表示文法中有三行产生式，其中 S、A、B 为非终结符，# 表示 ϵ 。

7.2 PDA 输入格式

当用户选择选项 4 时，程序会提示用户输入 PDA 相关信息。首先输入状态数和栈符号数，然后输入所有栈符号，再输入初始状态和初始栈符号，之后输入转移函数条数，并逐条输入转移函数。

程序中的转移函数输入格式为：

p input stackTop r pushString

其中，p 表示当前状态，input 表示读入符号，stackTop 表示当前栈顶符号，r 表示转移后的状态，pushString 表示压入栈中的字符串。如果输入符号或压栈字符串为空，可以使用#表示 ϵ 。代码中对该格式进行了提示，例如 0 b Z 0 BZ 表示在状态 0 读入 b，栈顶为 Z 时，转移到状态 0，并压入 BZ。

7.3 输出格式

程序输出采用分阶段展示方式。对于 CFG 化简，程序会依次输出原始文法、消除 ϵ 产生式后的文法、消除单产生式后的文法和消除无用符号后的文法。这样可以清楚观察文法在每一步变换后的变化情况。

输出标题包括：

Original grammar:

After removing epsilon-productions:

After removing unit-productions:

After removing useless symbols:

该输出顺序与程序 simplify() 函数中的处理顺序一致。

八、测试用例与运行结果

8.1 测试用例一：实验给定 CFG 样例

实验文档要求至少使用如下 CFG 样例进行程序正确性验证：

$S \rightarrow a|bA|B|ccD$

$A \rightarrow abB|\epsilon$

$B \rightarrow aA$

$C \rightarrow ddC$

$D \rightarrow ddd$

在程序中，选择选项 2 即可直接调用该内置样例。该样例中包含 ϵ 产生式 $A \rightarrow \epsilon$ ，包含单产生式 $S \rightarrow B$ ，同时 $C \rightarrow ddC$ 无法从开始符号 S 到达，属于无用符号相关内容。因此，该测试样例可以较好地验证程序是否能够完成消除 ϵ 产生式、消除单产生式和消除无用符号三个核心步骤。

程序运行后，首先输出原始文法，然后输出消除 ϵ 产生式后的文法。由于 A 可以推出 ϵ ，因此右部中含有 A 的位置会产生删除 A 后的新候选式。接着程序消除单

产生式，将通过单产生式间接得到的非单产生式补充到对应非终结符下。最后程序删除无用符号，使最终输出文法中不再包含对语言生成没有贡献的部分。

```
CFG Transformation Experiment
1. Input CFG and simplify
2. Use the given CFG sample
3. Convert the given PDA to CFG and simplify
4. Input PDA, convert to CFG and simplify
Choose: 2

Original grammar:
S->a|bA|B|ccD
A->abB|#
B->aA
D->ddd
C->ddC

After removing epsilon-productions:
S->a|b|bA|B|ccD
A->abB
B->a|aA
D->ddd
C->ddC

After removing unit-productions:
S->a|b|bA|ccD|aA
A->abB
B->a|aA
D->ddd
C->ddC

After removing useless symbols:
S->a|b|bA|ccD|aA
A->abB
B->a|aA
D->ddd
```

8.2 测试用例二：实验给定 PDA 样例

实验文档要求至少使用给定 PDA 进行程序正确性验证。该 PDA 为：

$M = (\{q_0, q_1\}, \{a, b\}, \{B, z_0\}, \delta, q_0, z_0, \Phi)$

$\delta(q_0, b, z_0) = \{(q_0, Bz_0)\}$

$\delta(q_0, b, B) = \{(q_0, BB)\}$

$\delta(q_0, a, B) = \{(q_1, \epsilon)\}$

$\delta(q_1, a, B) = \{(q_1, \epsilon)\}$

$\delta(q_1, \epsilon, B) = \{(q_1, \epsilon)\}$

$\delta(q_1, \epsilon, z_0) = \{(q_1, \epsilon)\}$

在程序中，选择选项 3 即可调用该内置 PDA 样例。程序首先根据 PDA 状态和栈符号构造变量，并输出变量含义。例如，程序将 A、B、C、D 等大写字母映射为对应的 $[q_i, \text{栈符号}, q_j]$ 形式。然后程序根据 PDA 转移函数生成 CFG 产生式。

生成原始 CFG 后，程序继续调用 CFG 化简算法，依次输出消除 ϵ 产生式、消除单产生式和消除无用符号后的结果。该测试可以验证程序是否能够把 PDA 的识别能力转化为 CFG 的生成能力，并进一步验证两个实验任务之间的衔接是否正确。

```

CFG Transformation Experiment
1. Input CFG and simplify
2. Use the given CFG sample
3. Convert the given PDA to CFG and simplify
4. Input PDA, convert to CFG and simplify
Choose: 3

Variable meanings in PDA-to-CFG construction:
A=[q0,B,q0] B=[q0,B,q1] C=[q0,z0,q0] D=[q0,z0,q1]
E=[q1,B,q0] F=[q1,B,q1] G=[q1,z0,q0] H=[q1,z0,q1]

Original grammar:
S->C|D
A->bAA|bBE
B->bAB|bBF|a
C->bAC|bBG
D->bAD|bBH
F->a|#
H->#

After removing epsilon-productions:
S->C|D
A->bAA|bBE
B->bAB|bB|bBF|a
C->bAC|bBG
D->bAD|bB|bBH
F->a

After removing unit-productions:
S->bAC|bBG|bAD|bB|bBH
A->bAA|bBE
B->bAB|bB|bBF|a
C->bAC|bBG
D->bAD|bB|bBH
F->a

After removing useless symbols:
S->bB
B->bB|bBF|a
F->a

```

8.3 测试用例三：手动输入 CFG 并化简

为了验证程序对用户自定义 CFG 输入的支持，可以选择选项 1，输入自定义文法。例如：

Input number of production lines: 4

S->AB|CD|a

A->E|#

B->bB|#

C->F

该测试主要用于检查程序是否能够正确解析用户输入的产生式，是否能够识别#表示的空串，以及是否能够对手动输入的文法完成完整化简过程。

程序输出结果仍然按照原始文法、消除 ϵ 产生式后文法、消除单产生式后文法、消除无用符号后文法的顺序展示。通过该测试可以看出，程序并不只依赖内置样例，而是能够处理一般形式的 CFG 输入。

```

CFG Transformation Experiment
1. Input CFG and simplify
2. Use the given CFG sample
3. Convert the given PDA to CFG and simplify
4. Input PDA, convert to CFG and simplify
Choose: 1
Input number of production lines: 4
Input productions line by line, for example: S->a|bA|#
Use # for epsilon. The program can also recognize epsilon if your terminal supports it.
S->AB|CD|a
A->E|#
B->bB|#
C->F

Original grammar:
S->AB|CD|a
A->E|#
B->bB|#
C->F

After removing epsilon-productions:
Z->#|S
S->B|A|AB|CD|a
A->E
B->b|bB
C->F

After removing unit-productions:
Z->#|AB|CD|a|b|bB
S->AB|CD|a|b|bB
B->b|bB

After removing useless symbols:
Z->#|a|b|bB
B->b|bB

Press any key to continue . . . |

```

8.4 测试用例四：手动输入 PDA 并转换 CFG

为了验证程序对自定义 PDA 输入的支持，可以选择选项 4。程序会依次要求用户输入状态数、栈符号数、栈符号集合、初始状态、初始栈符号以及转移函数。输入完成后，程序会自动生成变量含义、原始 CFG 和化简后的 CFG。

该测试说明程序不仅能够处理实验给定 PDA 样例，还能够支持用户输入新的 PDA。由于程序中的变量命名依赖有限的大写字母集合，因此程序对状态数和栈符号数设置了限制，要求 $\text{states}^2 * \text{stack_symbols} \leq 25$ 。这样可以避免变量数量超过可用命名范围，从而保证程序输出可读、运行稳定。

```

CFG Transformation Experiment
1. Input CFG and simplify
2. Use the given CFG sample
3. Convert the given PDA to CFG and simplify
4. Input PDA, convert to CFG and simplify
Choose: 4
This PDA is accepted by empty stack.
States are numbered from 0 to n-1.
Input state count and stack-symbol count: 2 2
Input stack symbols as one word, e.g. BZ means B and Z: BZ
Input start state number and start stack symbol: 0 Z
Input transition count: 6
Transition format: p input stackTop r pushString
Use # for epsilon. Example: 0 b Z 0 BZ
0 b Z 0 BZ
0 b B 0 BB
0 a B 1 #
1 a B 1 #
1 # B 1 #
1 # Z 1 #

```



```

Variable meanings:
A=[q0,B,q0]   B=[q0,B,q1]
C=[q1,B,q0]   D=[q1,B,q1]
E=[q0,Z,q0]   F=[q0,Z,q1]
G=[q1,Z,q0]   H=[q1,Z,q1]

Original grammar:
S->E|F
A->bAA|bBC
B->bAB|bBD|a
D->a|#
E->bAE|bBG
F->bAF|bBH
H->#

After removing epsilon-productions:
S->E|F
A->bAA|bBC
B->bAB|bB|bBD|a
D->a
E->bAE|bBG
F->bAF|bB|bBH

After removing unit-productions:
S->bAE|bBG|bAF|bB|bBH
A->bAA|bBC
B->bAB|bB|bBD|a
D->a
E->bAE|bBG
F->bAF|bB|bBH

After removing useless symbols:
S->bB
B->bB|bBD|a
D->a

Press any key to continue . . .

```

九、实验结果分析

1. 从 CFG 样例测试结果可以看出，程序能够正确完成消除 ϵ 产生式、消除单产生式和消除无用符号三个步骤。程序先计算可空非终结符集合，再通过单产生式可达关系完成替换，最后删除不可生成或不可达符号，使最终文法更加简洁。
2. 从 PDA 转换 CFG 测试结果可以看出，程序能够根据 PDA 状态、栈符号和转移规则构造等价 CFG。转换完成后，程序继续调用 CFG 化简算法，实现了“PDA 转换 CFG—CFG 化简”的完整处理流程。
3. 从程序功能上看，本程序不仅支持实验给定样例验证，还支持用户手动输入 CFG 和 PDA，具有一定的通用性与扩展性。同时，程序能够分阶段输出各步骤结果，方便观察算法执行过程与文法变化情况。

十、程序不足与改进方向

本程序虽然完成了实验要求中的主要功能，但仍存在一些可以改进的地方。

1. **变量命名仍有一定限制：**程序主要使用单个大写字母表示非终结符，在 PDA 状态数或栈符号较多时，变量数量可能不够。后续可以改为字符串形式，如 $[q0, B, q1]$ 。
2. **输入输出方式还可以优化：**目前程序主要依靠控制台输入输出，适合实验演

- 示，但处理较大文法时不够方便。后续可以增加文件读取和结果保存功能。
3. **输出格式还可以进一步改进：**当前结果以普通文本显示，后续可以增加表格化输出，或对每一步删除、替换的产生式进行说明，使结果更清晰。

十一、实验总结

通过本次实验，我们进一步理解了上下文无关文法化简与下推自动机转换文法的相关理论，同时也提高了使用 C 语言实现形式语言算法的能力。

1. **CFG 化简部分**，我们掌握了消除 ϵ 产生式、消除单产生式以及消除无用符号的基本思想。实验过程中，需要先求出可空非终结符集合，再建立非终结符之间的可达关系，最后判断符号是否具有生成能力和可达性。通过这些算法的实现，我们进一步理解了文法化简的整体流程以及各部分之间的联系。
2. **在 PDA 转换 CFG 部分**，我们进一步理解了下推自动机与上下文无关文法之间的等价关系。程序通过构造文法变量，将 PDA 中的状态转移与栈操作转换为 CFG 中的产生式推导过程，从而实现了由 PDA 到 CFG 的自动转换。通过这一过程，我们对形式语言理论中的自动机与文法之间的对应关系有了更加直观的认识。
3. **在程序设计方面**，本实验采用模块化结构实现，将程序划分为 CFG 输入解析、文法化简、PDA 转换 CFG 以及主菜单交互等多个部分。程序不仅完成了实验要求中的固定样例验证，还设计了四个功能选项，支持手动输入 CFG、使用给定 CFG 样例、使用给定 PDA 样例以及手动输入 PDA 进行转换与化简，提高了程序的通用性与扩展性。
4. 此外，在实验过程中，我们还进一步熟悉了 C 语言中的字符串处理、结构体设计、递归调用以及数组管理等内容。通过不断测试和调试程序，也提高了分析问题和解决问题的能力。

总体来说，本次实验不仅帮助我们巩固了形式语言与自动机课程中的核心理论，也提升了算法实现与程序设计能力，为后续学习编译原理、语法分析等相关内容打下了良好基础。